Computer Vision Practical Report:

OUTLINE USING CANNY FUNCTION

1. AIM

To read an input color image using OpenCV, detect and outline the edges of objects using the Canny edge detection function, display both the original image and Canny edge image side-by-side using Matplotlib, and save the outlined image to disk.

2. PROCEDURE

- Load the input image using cv2.imread('E3-IMG.png').
- Convert the image from BGR to RGB using cv2.cvtColor().
- Convert the image to grayscale using cv2.cvtColor() with cv2.COLOR_BGR2GRAY.
- Apply Canny edge detection using cv2.Canny() with suitable threshold values.
- Create a Matplotlib figure of size 10x5.
- Display the original image and Canny edge image side-by-side using plt.subplot().
- Set titles as **'Original Image'** and **'Canny Edge Image'**.
- Disable axes using plt.axis('off').
- Display the images using plt.show().
- Save the Canny edge image as **canny_image.png** using cv2.imwrite()

3. PROGRAM CODE

```
import cv2
import matplotlib.pyplot as plt

img_bgr = cv2.imread('E3-IMG.png')
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
gray_img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray_img, threshold1=100, threshold2=200)
```

```
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title('Canny Edge Outline')
plt.axis('off')

plt.tight_layout()
plt.show()
```

4. OUTPUT

5. RESULT

The Python script executed successfully. The input image 'E3-IMG.png' was loaded, converted into a Canny edge image.